Introduction to Container Technologies

Introduction to DevOps

Joonas Ryynänen

LUT University

CONTENTS

- Introduction to software containerization
- Important terms and operations
- Containerization and cloud environments

Introduction

- In order to run a software, two things are needed:
  - The executable software artefact that is produced from the source code of the software
  - A suitable runtime environment for the software executable
- Examples:
  - Node.js-based web application requires the Node runtime environment
  - Java Spring application requires the Java runtime environment
  - A static web page requires a web server as its runtime environment
- Version control systems are used to synchronize and save the work regarding the software source code (and the artefact that is built from it)
- However, the runtime environment of the software can’t be synchronized via traditional version control -> this is where software containers come into the picture

Software containerization

- Containers offer a way to package both the software and its runtime environment

- They are often compared to virtual machines
  - Virtual machines are an abstraction of physical hardware
    - To run an application in a VM, an operating system, the application and the runtime environment is required
  - Containers are an abstraction of the app layers
    - Multiple containers can run on the same machine (for example a VM), also they can share the OS kernel with each other

Important terms regarding containerization

- Image: a read-only “blueprint” that represent the application and its runtime environment
  - Contains the source code, libraries, dependencies, tools and other required files in order to run the application
  - Instructions for creating a container
- Dockerfile:
  - A special file that is used to describe images
- Container: a runnable instance of an image
  - Images become containers when they are run

Important operations regarding containerization

- Building an image
  - Dockerfile is used for describing images
  - Ready-made images can be used as a starting point (eg. Node.js: https://hub.docker.com/_/node)

- Pushing an image
  - Images can be pushed to a remote repository (similar as to version control repositories)
    - Dockerhub, Azure Container Registry, GitLab Container Registry etc.
  - The image can then be obtained by accessing the repository

- Running a container
  - Docker CLI can be used to create containers from images
    - For example, command “docker run -d -p 80:80 docker/getting-started” runs the image “docker/getting-started” from the Dockerhub (https://hub.docker.com/_/getting-started)
    - The meaning of the options “-d” and -p” can be found here: https://docs.docker.com/get-started/#start-the-tutorial

Containerized applications and cloud environments

- Major cloud providers offer services that allow to run containerized applications:
  - AWS ECR
  - Google Cloud Run
  - Azure Web App Service for Containers

- Cloud services are discussed more in detail in the next part of the course

Bonus: container orchestration

- Large software solutions can be divided into multiple containers
- Example:
  - web API container
  - database container
  - client-side application container

-> it becomes hard to manage the containers manually

- Container orchestration solutions offer help regarding this problem
  - Docker Compose (compare Dockerfile vs. docker-compose.yml)
  - Docker Swarm
  - Kubernetes

References

- Docker: Empowering app development for developers. https://www.docker.com/

THANK YOU!


LUT
University